A. Simple Merge

Time limit: 2 s · Memory limit: 256 MB

You are given T sets of strings. For every set print its strings ordered by length: shorter strings first. If two strings have the same length, they must keep their relative order from the input — the sorting must be **stable**.

Input

The first line contains one integer T ($1 \le T \le 1000$) — the number of sets.

Each of the next T lines describes one set and contains k_i strings separated by single spaces ($1 \le k_i \le 10^5$). Every string consists of lowercase Latin letters and has length between 1 and 50 inclusive.

The total number of strings over all sets does not exceed $2 \cdot 10^5$, and the total length of all strings does not exceed 10^6.

Output

Print exactly T lines. The i-th line must contain all k_i strings of the i-th set separated by single spaces, ordered by non-decreasing length. Strings of equal length must be printed in the same relative order in which they occur in the input.

Note

The intended solution is merge sort — a stable $O(k \log k)$ sorting algorithm. Note that quick sort is **not** stable, so it cannot be used here without an additional tie-breaking key.

Examples

Input 1

```
3
ab cd e j asd ljffg df
a a b b c c
xy yx zxy zx xzy xxx
```

Output 1

```
e j ab cd df asd ljffg
a a b b c c
xy yx zx zxy xzy xxx
```

Input 2

```
4
aabc ddfd fbbe f baaad
fddf badcf aedbe cdb ae bbd
ada bc eaced dadab b aabc fbee
bedaf edde fcaa adbf beec
```

Output 2

```
f aabc ddfd fbbe baaad
ae cdb bbd fddf badcf aedbe
b bc ada aabc fbee eaced dadab
edde fcaa adbf beec bedaf
```

B. Merge Two Sorted Arrays

Time limit: 1 s · Memory limit: 256 MB

You are given two arrays of integers, a and b. Both of them are already sorted in non-decreasing order.

Merge them into one sorted array that contains all elements of a and all elements of b.

Input

The first line contains one integer n ($1 \le n \le 10^5$) — the size of a.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^5$) separated by single spaces. The array is sorted: $a_i \le a_{i+1}$.

The third line contains one integer m ($1 \le m \le 10^5$) — the size of b.

The fourth line contains m integers $b_1, b_2, \ldots, b_m$ ($1 \le b_i \le 10^5$) separated by single spaces. The array is sorted: $b_i \le b_{i+1}$.

Output

Print $n + m$ integers separated by single spaces — all elements of both arrays in non-decreasing order.

Note

Both arrays are already sorted, so there is no need to sort anything. Walk along the two arrays with two indices and always take the smaller of the two current elements. This is exactly the **merge** step of merge sort, and it works in $O(n + m)$.

Do not forget the tail: when one array runs out, all remaining elements of the other one still have to be printed.

The answer always contains exactly $n + m$ numbers.

Examples

Input 1

```
5
4 5 6 7 10
5
2 4 6 7 8
```

Output 1

```
2 4 4 5 6 6 7 7 8 10
```

Input 2

```
3
1 4 7
1
6
```

Output 2

```
1 4 6 7
```

C. Sorter

Time limit: 1 s · Memory limit: 256 MB

Almas loves to sort numbers with specific rules that he comes up with himself. Today he wants to sort rows, each row contain m integer elements. He decided to sort rows in decreasing order of their sum, i.e. the row with larger sum will be the first. In case the sums are equal, then he order rows lexicographically, i.e. The row with least first element will be the first, if first elements are equal, then compare second element, third element, etc. He successfully finished sorting. He wanted to put the result in the closet, but accidentally put it in the shredder and divided everything into rows again. Help him sort it again.

Input

The first line contains two numbers, n and m the number of rows and number of elements in a row. The next n lines contain m numbers, elements of each row.

- $1 \leq n, m \leq 500$
- $0 \leq row[i] \leq 5000$

Output

Print rows in desired sorted order.

Examples

Input 1

```
5 3
1 2 3
1 2 4
1 3 2
6 0 0
2 3 2
```

Output 1

```
1 2 4
2 3 2
1 2 3
1 3 2
6 0 0
```

Input 2

```
2 3
16 3 18
5 10 8
```

Output 2

```
16 3 18
5 10 8
```

D. Count the Inversions

Time limit: 2 s · Memory limit: 256 MB

Aidana is checking how messy a list of numbers is.

She calls a pair of positions (i, j) an **inversion** if $i < j$ and $a_i > a_j$ — that is, a larger number stands before a smaller one.

A sorted array has no inversions at all, and the more inversions an array has, the further it is from being sorted. Help Aidana count them.

Input

The first line contains one integer n ($1 \le n \le 10^5$) — the length of the array.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) separated by single spaces.

Output

Print one integer — the number of inversions in the array.

Note

In the first example the pair $(1, 2)$ is an inversion, because $2 > 1$.

In the second example the array is $[3, 1, 4, 1, 5]$ and the inversions are $(1, 2)$, $(1, 4)$ and $(3, 4)$.

Note that equal elements do **not** form an inversion: the condition is strictly $a_i > a_j$.

The intended solution is merge sort: while merging two sorted halves, every element taken from the right half is smaller than all remaining elements of the left half, which gives the number of inversions in $O(n \log n)$. A straightforward double loop over all pairs is $O(n^2)$ and will not fit into the time limit.

The answer can be as large as $n(n-1)/2 \approx 5 \cdot 10^9$, so a 32-bit integer type is **not** enough.

Examples

Input 1

```
2
2 1
```

Output 1

```
1
```

Input 2

```
5
3 1 4 1 5
```

Output 2

```
3
```

E. Squares of a Sorted Array

Time limit: 2 s · Memory limit: 256 MB

Nurlan has an array of integers sorted in **non-decreasing** order. Some of the numbers may be negative.

He squares every element and wants to see the result sorted in non-decreasing order as well. Squaring breaks the order: -4 stands before 3, but 16 is greater than 9.

Print the squares of all elements in non-decreasing order.

Input

The first line contains one integer n ($1 \le n \le 2 \cdot 10^5$) — the length of the array.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($-10^4 \le a_i \le 10^4$) separated by single spaces. The array is sorted: $a_i \le a_{i+1}$ for every i.

Output

Print n integers separated by single spaces — the squares of the elements in non-decreasing order.

Note

In the first example the array is $[-4, -1, 0, 3, 10]$, the squares are $[16, 1, 0, 9, 100]$, and sorted they give $[0, 1, 9, 16, 100]$.

Sorting the squares directly would work, but there is a faster way that uses **merging**. Split the array at the point where the sign changes. Squares of the negative part decrease from left to right, squares of the non-negative part increase. These are two sorted sequences, so they can be merged in $O(n)$ — the same merge step that merge sort is built from. It is convenient to take the largest square at each step, comparing the two ends of the array, and to fill the answer from the end.

Examples

Input 1

```
5
-4 -1 0 3 10
```

Output 1

```
0 1 9 16 100
```

Input 2

```
4
-7 -3 2 11
```

Output 2

```
4 9 49 121
```